Unit 3 : Servlet

3.1 Fundamentals of Java Servlet Programming

➢ Introduction to Servlet

- A **Servlet** is a **server-side Java program** that runs inside a web server such as Apache Tomcat and is used to handle **client requests (from browser)** and generate **dynamic responses (HTML output)**.
- In simple words:
  A servlet acts like a **bridge between browser and database/business logic**.

➢ Need for Servlets

- Before servlets, **CGI (Common Gateway Interface)** was used, but it had problems:
  - Slow performance (new process for each request)
  - High memory usage

Servlets solve these issues:

- Use **multithreading** (one instance handles many users)
- Faster and efficient
- Better memory management

➢ How Servlet Works (Architecture)

Flow Explanation:

1. User enters URL in browser
2. Browser sends request to server
3. Server (Tomcat) receives request
4. Server forwards request to Servlet
5. Servlet processes logic
6. Servlet sends response (HTML)
7. Browser displays output

Browser → Request → Server → Servlet → Response → Browser

➢ Servlet API (Core Packages)

Main Core Packages

1. javax.servlet

- **Basic servlet functionality (core interfaces & classes)**
- Important Classes / Interfaces:

  - **Servlet**
  - **GenericServlet**
  - **ServletRequest**
  - **ServletResponse**
  - **ServletConfig**
  - **ServletContext**
  - **RequestDispatcher**

Example Use:

import javax.servlet.*;

Used for:

- Lifecycle methods (init(), destroy())
- Request/Response handling (basic level)
- Server communication

2. javax.servlet.http

HTTP-specific servlet features (MOST IMPORTANT)

Important Classes:

- **HttpServlet**
- **HttpServletRequest**
- **HttpServletResponse**
- **HttpSession**
- **Cookie**

Example:

import javax.servlet.http.*;

Used for:

- Handling HTTP methods (doGet(), doPost())
- Session tracking
- Cookies
- Form data handling

3. javax.servlet.annotation

Used for annotations (modern approach)

Important Annotation:

@WebServlet

Example:

import javax.servlet.annotation.WebServlet;

@WebServlet("/HelloServlet")

Purpose:

- No need for web.xml
- Easy URL mapping

4. javax.servlet.tagext (Advanced)

Used for **JSP custom tags**

Contains:

- Tag interfaces
- Tag support classes

Mostly used in:

JSP (not basic servlet programs)

➢ Important Interfaces and Classes in Servlet

- Servlet programming is based on a **set of interfaces and classes** provided by the packages:

1. Servlet (Interface)

This is the main interface of all servlets

Key Methods:

1. **init()** → Called once when servlet starts
2. **service()** → Handles requests (again & again)
3. **destroy()** → Called when servlet is removed

Example:

- init() = Starting engine
- service() = Driving
- destroy() = Stop engine

Defines **life cycle of servlet**

2. ServletRequest

Represents client request

Common Methods:

- getParameter()
- getParameterValues()
- getAttribute()

Used to read **form data / user input**

3. ServletResponse

Represents response sent to client

Methods:

- getWriter()
- setContentType()

Used to send output to browser

4. ServletConfig

Provides configuration info of servlet

Methods:

- getInitParameter()
- getServletName()

5. ServletContext

Represents entire web application

Methods:

- getInitParameter()
- getAttribute()
- setAttribute()

Used for **sharing data across servlets**

6. RequestDispatcher

Used to forward/include request

Methods:

- forward()
- include()

Important Classes

1. GenericServlet

Implements Servlet interface

✓ Protocol independent
✓ Provides default implementation

You extend this when:

- You don’t need HTTP-specific features

2. HttpServlet (MOST IMPORTANT)

Extends GenericServlet

✓ Used for **HTTP-based applications**

Important Methods:

- doGet()
- doPost()
- doPut()
- doDelete()

Your programs use this class ✓

3. HttpServletRequest

Extends ServletRequest

✓ Used to get:

- Form data
- Parameters
- Headers

Example:

request.getParameter("name");

4. HttpServletResponse

Extends ServletResponse

✓ Used to send response

Example:

response.getWriter();

5. HttpSession

Used for **session tracking**

✓ Stores user data across multiple requests

Example:

HttpSession session = request.getSession();

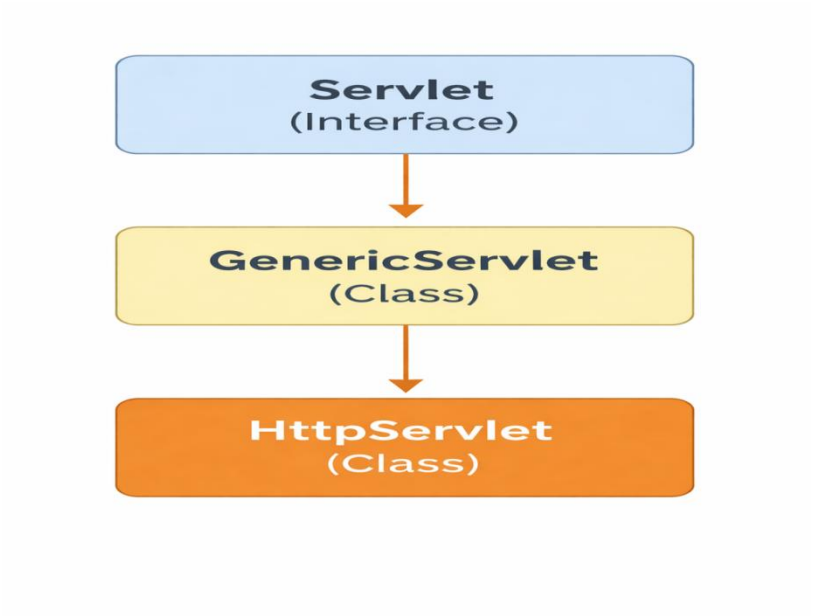
6. Cookie

Used for **client-side storage**

Example:

Cookie c = new Cookie("name","value");

Relationship Diagram



Summary Table :

Component	Type	Purpose
Servlet	Interface	Defines lifecycle
GenericServlet	Class	Base implementation
HttpServlet	Class	HTTP-based servlet
ServletRequest	Interface	Client request
ServletResponse	Interface	Server response
HttpServletRequest	Interface	HTTP request
HttpServletResponse	Interface	HTTP response

Key Points

- Servlet is the **root interface**
- GenericServlet simplifies implementation
- HttpServlet is **most used**
- Request & Response handle communication
- Used in all web applications

➢ Advantages of Servlets

- High performance (multithreading)
- Platform independent
- Secure (Java-based)
- Scalable for large applications

➢ Limitations of Servlets

- More coding required
- UI creation is difficult (HTML inside Java)
- Requires setup in tools like Eclipse IDE for Enterprise Java and Web Developers

3.2 A Simple Java Servlet

1. Basic Example Program to display message on browser by using servlet

```
package com.demo;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/HelloServlet")
public class HelloServlet extends HttpServlet
{

protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws IOException
{
response.setContentType("text/html");
PrintWriter out = response.getWriter();

out.println("<h1>Hello Yogita! Servlet is working!</h1>");
}
}
```

Program Explanation (Line by Line)

1. package com.demo;

- This defines the **package name**
- It helps to organize your Java files properly
  Think of it like a **folder for your class**

2. import java.io.*;

- Imports input-output classes
- Needed for **PrintWriter** (to print output)

3. import javax.servlet.*;

- Imports basic servlet classes
- Includes lifecycle-related classes

4. import javax.servlet.http.*;

- Imports HTTP-specific classes
- Required for:
  - HttpServlet
  - HttpServletRequest
  - HttpServletResponse

5. import javax.servlet.annotation.*;

- Used for annotations like @WebServlet
- Helps in mapping URL to servlet

6. @WebServlet("/HelloServlet")

- This is **URL mapping**
- When user types:

http://localhost:8080/ProjectName/HelloServlet

➞This servlet will run

7. public class HelloServlet extends HttpServlet

- Creating a servlet class
- extends HttpServlet means:
- This class becomes a **web servlet**

8. protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException

- doGet() handles **GET request** (from browser)
- Automatically called when user opens URL

Parameters:

- request → gets data from browser
- response → sends data to browser

9. response.setContentType("text/html");

- Tells browser:
  "I am sending HTML content"
- So browser displays it properly

10. PrintWriter out = response.getWriter();

- Creates object to **print output on browser**
- Like System.out.println() but for web

11. out.println("<h1>Hello Yogita! Servlet is working!</h1>");

- Sends HTML response to browser
- <h1> → heading tag
- Output will appear as big heading

Final Output

When you run:

http://localhost:8080/ProjectName/HelloServlet

You will see:

Hello Yogita! Servlet is working!

3.3 Servlet Life Cycle

b) Explain Servlet Life Cycle and demonstrate its methods with example [10 Marks] [May 2025]

1. Introduction to Servlet Life Cycle

- The **Servlet Life Cycle** defines the stages through which a servlet passes from its **creation to destruction**.
- It is **controlled by the web server** like Apache Tomcat.

2. Phases of Servlet Life Cycle

- A servlet has **three main life cycle methods**:

Phase 1: Loading and Instantiation

- The servlet class is **loaded into memory** by the server
- An **object of the servlet is created**
- This happens **only once**

Loading can be:

- Automatically (first request)
- At server startup

Phase 2: Initialization – init() Method

✓ **Definition:**

The init() method is called **only once** after the servlet object is created.

✓ **Syntax:**

public void init() throws ServletException

✓ **Purpose:**

- Initialize resources
- Setup configuration
- Establish database connection

✓ **Key Points:**

- Called only once in lifetime
- Cannot be called again and again

Phase 3: Request Processing – service() Method

✓ **Definition:**

The service() method is called **every time a client sends a request**.

✓ **Syntax:**

public void service(ServletRequest req, ServletResponse res)

✓ **Working:**

- It is the **heart of servlet processing**
- It decides which method to execute:
    - doGet() → for GET request
    - doPost() → for POST request

In real applications, we override:

- doGet() or doPost() instead of service()

Phase 4: Destruction – destroy() Method

✓ **Definition:**

The destroy() method is called **once before the servlet is removed from memory**.

✓ **Syntax:**

public void destroy()

✓ **Purpose:**

- Release resources
- Close database connections
- Clean up memory

Life Cycle Diagram:

Detailed Flow

1. Servlet is loaded by server (Apache tomcat)
2. init() is called (once)
3. For each request:
    - service() executes
    - Calls doGet() / doPost()
4. When server stops:
    - destroy() is called

Example Program Demonstrating Life Cycle

```
package com.demo;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/LifeCycleServlet")
public class LifeCycleServlet extends HttpServlet
{

public void init()
{
System.out.println("Servlet Initialized");
}

protected void doGet(HttpServletRequest request,
HttpServletResponse response)
throws IOException
{

response.setContentType("text/html");
PrintWriter out = response.getWriter();

out.println("<h2>Service Method Executed</h2>");
out.close();
}

public void destroy()
{
System.out.println("Servlet Destroyed");
}
}
```

Output:

Service Method Executed

Line-by-Line Explanation

1. Package Declaration

package com.demo;

Defines package name where your servlet class is stored.

2. Import Statements

import java.io.IOException;
import java.io.PrintWriter;

Used for:

- IOException → handling input/output errors
- PrintWriter → sending response to browser

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

3. Servlet-related classes:

- ServletException → handles servlet errors
- @WebServlet → maps URL to servlet
- HttpServlet → base class for HTTP servlet

4. Annotation (URL Mapping)

@WebServlet("/LifeCycleServlet")

This connects your servlet to URL:

http://localhost:8080/LifeCycleDemo/LifeCycleServlet

5. Class Declaration

public class LifeCycleServlet extends HttpServlet {

Your servlet class:

- Inherits HttpServlet
- Can handle HTTP requests (GET, POST)

6. Servlet Life Cycle Methods

1. init() Method

```
public void init()
{
   System.out.println("Servlet Initialized");
}
```

Called **only once** when servlet is loaded first time
Used for:

- Initialization (DB connection, config etc.)

Output in console:

Servlet Initialized

2. doGet() Method (Service Method)

```
protected void doGet(HttpServletRequest request, HttpServletResponse
response)
      throws IOException {
```

Called **every time user sends request (GET)**

Set Response Type

response.setContentType("text/html");

Tells browser: response is **HTML**

Create PrintWriter

PrintWriter out = response.getWriter();

Used to send data to browser

Display Output

out.println("<h2>Service Method Executed</h2>");

Sends HTML response to browser

Messages can be seen in **console**

Example 3:

Create a HTML page to accept two numbers and write servlet to add given numbers and display result. [10] [May 2025]

1. Project Setup in Eclipse

Step 1: Create Dynamic Web Project

1. Open Eclipse
2. Go to **File → New → Dynamic Web Project**
3. Project Name: AdditionServletApp
4. Target Runtime: Select Apache Tomcat (configure if not added)
5. Click **Finish**

Step 2: Create HTML File

1. Right click project → **New → HTML File**
2. Name: index.html

✨HTML Code[index.html]

```
<!DOCTYPE html>
<html>
<head>
<title>Addition Form</title>
</head>

<body>
<h2>Add Two Numbers</h2>

<form action="add" method="post">
Enter Number 1: <input type="text"
name="num1"><br><br>
Enter Number 2: <input type="text"
name="num2"><br><br>
```

```
<input type="submit" value="Add">
</form>
</body>
</html>
```

Step 3: Create Servlet Class

1. Right click project → **New** → **Servlet**
2. Class Name: AddServlet
3. Package: com.example
4. Click **Finish**

✨Servlet Code (AddServlet.java)

```
package com.example;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/add")
public class AddServlet extends HttpServlet
{

protected void doPost(HttpServletRequest request,
HttpServletResponse response)
throws IOException
{

response.setContentType("text/html");
PrintWriter out = response.getWriter();

int num1 =
Integer.parseInt(request.getParameter("num1"));
int num2 =
Integer.parseInt(request.getParameter("num2"));

int result = num1 + num2;
```

```
out.println("<h2>Result: " + result + "</h2>");
}
}
```

4. Configure Server (Tomcat)

1. Go to **Servers tab**
2. Click **“No servers available” → Add new server**
3. Select **Apache Tomcat**
4. Browse and select Tomcat installation folder
5. Click **Finish**

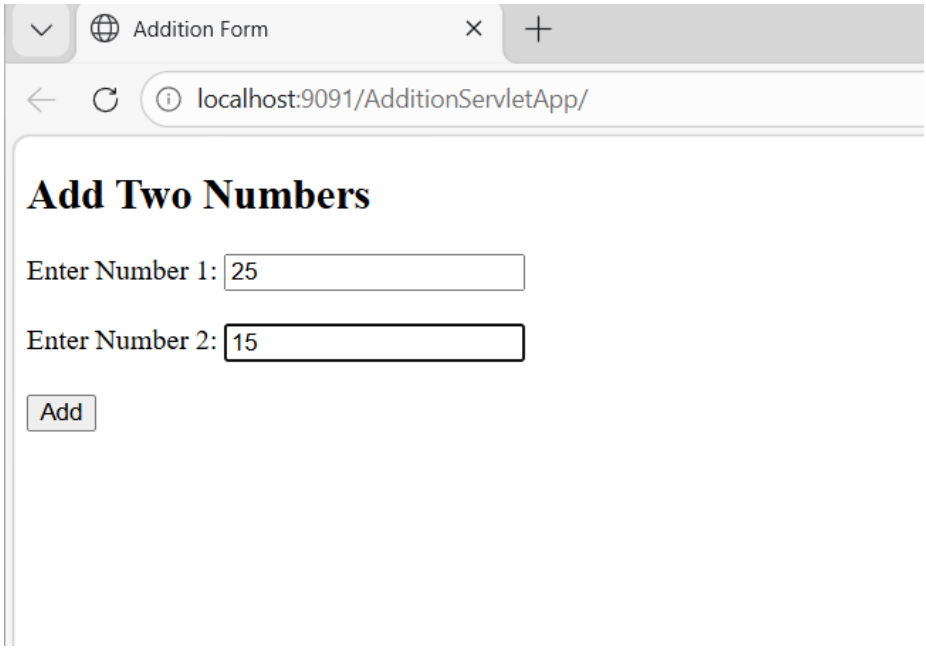
5. Run the Project

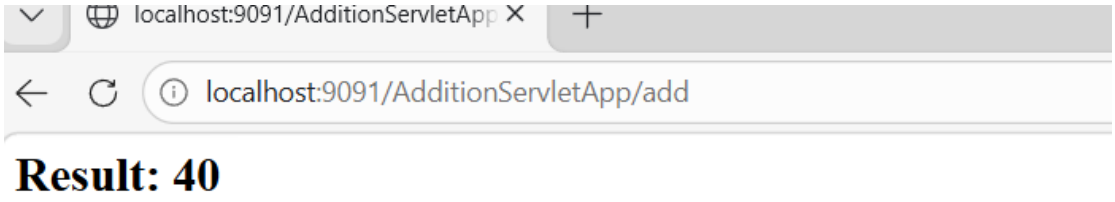
1. Right click project → **Run As → Run on Server**
2. Choose Tomcat → Finish

6. Output

Open browser:

http://localhost:8080/AdditionServletApp/index.html





Key Points

- HTML form sends data using **POST method**
- request.getParameter() is used to read input
- Integer.parseInt() converts string to int
- Servlet processes logic and returns response using PrintWriter

Example 4:
create a servlet which check username and password. if it is correct display message "Successfully login" else display appropriate error message.(without JDBC)
[Jan 2026] [10 marks]

Servlet Program: Login Validation (Without JDBC)

Objective

Create a servlet that:

- Checks username & password
- If correct → **"Successfully login"**
- If wrong → **Error message**

Step 1: Create Dynamic Web Project in Eclipse

1. Open Eclipse
2. Click: **File → New → Dynamic Web Project**
3. Enter Project Name: LoginApp
4. Select Target Runtime: **Apache Tomcat**
5. Click **Finish**

Step 2: Create HTML Login Page

Right click on project → New → HTML File → name: login.html

```
<!DOCTYPE html>
<html>
<head>
<title>Login Page</title>
</head>
<body>
<h2>Login Form</h2>

<form action="login" method="post">
Username: <input type="text" name="uname"><br><br>
Password: <input type="password" name="pass"><br><br>
<input type="submit" value="Login">
</form>
```

```
</body>
</html>
```

Step 3: Create Servlet Class

Right click → New → Servlet → name: LoginServlet

Servlet Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/login")
public class LoginServlet extends HttpServlet
{

protected void doPost(HttpServletRequest request, HttpServletResponse
response)
throws ServletException, IOException
{

response.setContentType("text/html");
PrintWriter out = response.getWriter();

// Get form data
String username = request.getParameter("uname");
String password = request.getParameter("pass");

// Hardcoded values (no database)
if(username.equals("admin") && password.equals("1234"))
{
out.println("<h2 style='color:green'>Successfully Login</h2>");
}
else
{
out.println("<h2 style='color:red'>Invalid Username or Password</h2>");
}
}
}
```

Step 4: Configure Server (Tomcat)

1. Go to **Servers tab**
2. Right click → New → Server
3. Select **Apache Tomcat v9**
4. Add your project → Finish

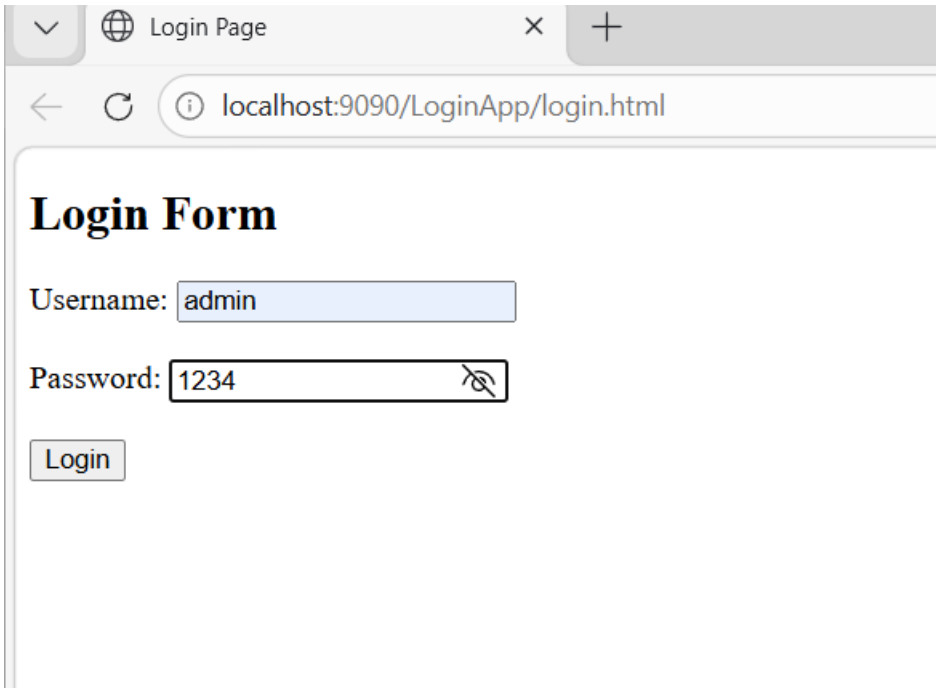
Step 5: Run the Project

1. Right click project → Run on Server
2. Browser opens
3. Open:

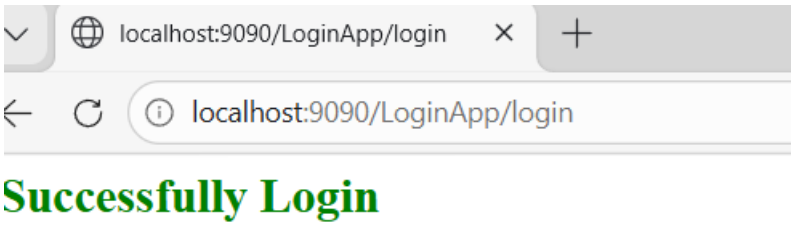
http://localhost:8080/LoginApp/login.html

Step 6: Test Output

✓ Correct Input:



Output:



Wrong Input:

Output: **Invalid Username or Password**

Program Explanation

- request.getParameter() → gets form data
- doPost() → handles POST request
- PrintWriter → prints output
- No JDBC → values are hardcoded

3.4 Developing and Deploying Servlets

What is a Servlet?

- A **Servlet** is a Java program that runs on a server and handles client requests (usually from a browser) and sends responses.

Steps to Develop a Servlet

1. Create a Java Class

Extend HttpServlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet
{

   public void doGet(HttpServletRequest request, HttpServletResponse
response)
          throws ServletException, IOException
{

       response.setContentType("text/html");
       PrintWriter out = response.getWriter();

       out.println("<h1>Hello Students!</h1>");
   }
}
```

2. Compile the Servlet

- Use servlet API jar (from Tomcat)
- Example:

```
javac HelloServlet.java
```

3. Configure Servlet (2 ways)

Method 1: Using Annotation

@WebServlet("/hello")

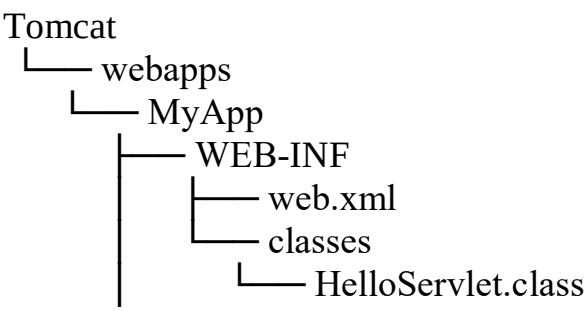
Method 2: Using web.xml

```
<servlet>
   <servlet-name>HelloServlet</servlet-name>
   <servlet-class>HelloServlet</servlet-class>
</servlet>

<servlet-mapping>
   <servlet-name>HelloServlet</servlet-name>
   <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

4. Deploy Servlet

Folder Structure (Tomcat)



5. Run the Servlet

Start server (Apache Tomcat)

Open browser:

http://localhost:8080/MyApp/hello

Example Output

Hello Students!

3.5 Working with Cookies

What are Cookies?

- Cookies are **small pieces of data stored in the user's browser**.
- Used to:
  - Remember user login
  - Store preferences
  - Track sessions

Types of Cookies

1. **Session Cookie** (temporary)
2. **Persistent Cookie** (stored for long time)

Creating a Cookie

```
Cookie cookie = new Cookie("username", "Yogita");
response.addCookie(cookie);
```

Reading Cookies

```
Cookie[] cookies = request.getCookies();

for(Cookie c : cookies)
{
   if(c.getName().equals("username"))
{
      out.println("Welcome " + c.getValue());
   }
}
```

Setting Cookie Expiry

```
cookie.setMaxAge(60*60); // 1 hour
```

Complete Example of Cookie:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/CookieExample")
public class CookieExample extends HttpServlet
{

protected void doGet(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException
{

response.setContentType("text/html");
PrintWriter out = response.getWriter();

// Create Cookie
Cookie c = new Cookie("name", "Yogita");
response.addCookie(c);

// Read Cookies
Cookie[] cookies = request.getCookies();

out.println("<html><body>");

if(cookies != null)
{
for(Cookie ck : cookies)
{
out.println(ck.getName() + " : " + ck.getValue() +
"<br>");
}
}
else
{
out.println("No cookies found");
}

out.println("</body></html>");
```

```
}
}
```

Advantages of Cookies

✓ Simple to use
✓ Stores user data
✓ Improves user experience

Limitations

✗Small size (4KB)
✗Security risk
✗Can be disabled by user

Real-Life Example

When you login to a website and it says
"Remember Me"
➞That is done using cookies

Q.3(b) Define Session Management using Cookies (10 Marks)

[January 2026]

Answer:

Definition: Session Management

- Session management using cookies is a method to maintain user data between multiple requests by storing small pieces of information (called cookies) in the user's browser.
- HTTP is a **stateless protocol**, so the server cannot remember the user. Cookies help to maintain the session.

What is Session?

- A session is a continuous interaction between user and server for a limited time.
- Example: Login to a website and browsing pages without logging again.

What is Cookie?

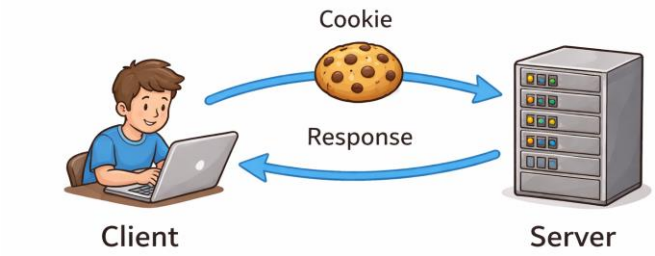
A cookie is a small text file stored in the browser which contains user information like:

- Username
- User ID
- Login status

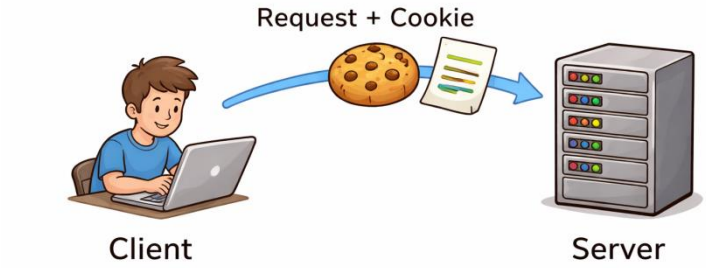
Working of Session using Cookies:

1. User sends request to server
2. Server validates user (login)
3. Server creates cookie (username/userID)
4. Cookie is stored in browser
5. Browser sends cookie with every request
6. Server identifies user using cookie

Diagram:



Next Request:
Client → Request + Cookie → Server



Example (Servlet Code):

Creating Cookie:

```
Cookie c = new Cookie("user","Yogita");
response.addCookie(c);
```

Reading Cookie:

```
Cookie[] cookies = request.getCookies();
for(Cookie c : cookies)
{
   if(c.getName().equals("user"))
```

```
{
        out.println("Welcome " + c.getValue());
    }
}
```

Advantages:

✓ Easy to implement
✓ Maintains session
✓ Improves user experience

Disadvantages:

✗Limited size (4KB)
✗Less secure
✗Can be disabled

Conclusion:

Cookies help in session management by storing user information in the browser, allowing the server to recognize users and provide continuous service.